



OASIS Security Report

File Upload Vulnerabilities Security Analysis

Date: 2026-06-12 01:34:07 — **Model:** llama3.2:3b

Vulnerability: File Upload Vulnerabilities

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **8** file(s) for File Upload Vulnerabilities.

[Return to table of contents](#)

Statistics

Files analyzed: 8 **Total findings:** 8

Critical



4

High		3
Medium		1
Low		0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.627
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.626
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.625
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.596
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.594
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.591
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.578
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.560

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.627

Source lines: 1-39

► Finding 1.1: Unrestricted File Upload (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw));
```

Explanation

The call() function allows the attacker to execute arbitrary Solidity code, including file uploads.

Impact

Remote code execution and web shell deployment.

Entry point

/withdrawFunds(uint256 _weiToWithdraw)

Execution path

```
POST /withdrawFunds <addr> <amount>
require(...)
call.value(_weiToWithdraw)()
...
```

HTTP methods

POST

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- Send malicious Solidity code as part of the amount

Example payloads

- maliciousSolidityCode

Conditions

Attacker can reach POST /withdrawFunds while authenticated.

Remediation

Validate file types and content before executing user-supplied code.

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.626

Source lines: 1-85

► Finding 2.1: Unrestricted file upload (Critical)

Vulnerable code

```
function ETH_VAULT(address _log) public { TransferLog = Log(_log); }
```

Explanation

The contract allows any address to set the `TransferLog` variable without validation.

Impact

Remote code execution via arbitrary file uploads.

Entry point

/ETH_VAULT(address)

Execution path

```
GET /ETH_VAULT(0x1234567890123456789012345678901234567890) => TransferLog = Log(_log)
POST /ETH_VAULT(0x1234567890123456789012345678901234567890) => TransferLog = Log(_log)
```

HTTP methods

GET, POST

Parameters

_log

Exploitation steps

- Send a malicious file via GET or POST request.

Example payloads

- `malicious_file.txt`

Conditions

No authentication required.

Remediation

Implement validation and sanitization for the `\_log` parameter.

Secure example

```
pragma solidity ^0.8.0;  
contract SecureETHVAULT { mapping (address => uint) public balances; ... }
```

File 3: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.625

Source lines: 1-35

► Finding 3.1: Unrestricted File Upload (High)

Vulnerable code

```
bool res = msg.sender.call.value(amount()); credit[msg.sender]-=amount;
```

Explanation

The contract allows any address to upload files without validation.

Impact

Remote code execution and denial of service attacks are possible.

Entry point

/withdraw(uint amount)

HTTP methods

POST

Parameters

amount

Exploitation steps

- User sends a large file to trigger a denial of service attack.
- Attacker uploads malicious code as a file and executes it.

Example payloads

- `malicious-executable-file.tar.gz`

Conditions

The contract does not validate the file type or content.

Remediation

Validate file types and content before uploading files.

Secure example

```
bool res = msg.sender.call.value(amount()) { credit[msg.sender]-=amount; }  
  
// Validate file extension
```

```
string filename = msg.data.toString();  
if (filename.endsWith('.exe') || filename.endsWith('.bat')) {  
    revert();  
}
```

File 4: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.596

Source lines: 1-42

► Finding 4.1: File Upload Vulnerability (Critical)

Vulnerable code

```
function setReward() public payable {  
    require (!claimed);  
    require(msg.sender == owner);  
    owner.transfer(reward);  
    reward = msg.value;  
}
```

Explanation

The `owner.transfer(reward)` line allows an attacker to execute arbitrary system commands as the contract owner.

Impact

Remote code execution

Entry point

/transfer

Execution path

```
msg.sender -> owner -> msg.sender transfer(reward)
```

HTTP methods

POST

Parameters

reward

Exploitation steps

- Send a large file to the `/setReward` endpoint

Example payloads

- A PHP script

Conditions

Contract owner has sufficient funds and permissions

Remediation

Use `transfer` instead of `owner.transfer` to prevent code injection

Secure example

```
function setReward() public payable {  
  require (!claimed);  
  require(msg.sender == owner);  
  deposits(reward);  
  reward = msg.value;  
}
```

File 5: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.594

Source lines: 1-31

► Finding 5.1: Unrestricted File Upload (Critical)

Vulnerable code

```
address owner = msg.sender;  
receiver.transfer(amount);
```

Explanation

The `owner` variable is set directly from the message sender without validation.

Impact

Remote code execution and data corruption

Entry point

/transfer

Execution path

```
msg.sender -> owner -> receiver -> tx.origin
```

HTTP methods

POST

Parameters

owner, receiver

Exploitation steps

- Sender sends a malicious contract to execute

Example payloads

- `malicious smart contract code`

Conditions

`tx.origin == owner`

Remediation

Validate file types and content, use secure upload directories.

Secure example

```
address owner = tx.origin;  
receiver.transfer(amount);
```

File 6: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.591

Source lines: 126-467

► Finding 6.1: Potential File Upload Vulnerability (Medium)

Vulnerable code

```
function changeRequirement(uint _newRequired) onlymanyowners(sha3(msg.data))  
external { if (_newRequired > m_numOwners) return; ... }
```

Explanation

The `\_newRequired` parameter is not validated, allowing an attacker to increase the number of required confirmations beyond the intended limit.

Impact

An attacker could force a higher-than-expected confirmation threshold, leading to unnecessary delays in transaction execution.

Entry point

/changeRequirement

Execution path

```
POST /changeRequirement (GET /) -> check m_numOwners  
POST /changeRequirement (POST /) -> update m_required
```

HTTP methods

POST

Parameters

_newRequired

Exploitation steps

- Send a POST request to /changeRequirement with an excessive value for _newRequired

Conditions

m_numOwners is greater than 0 and msg.sender has confirmed the operation

Remediation

Validate the `\_newRequired` parameter using a secure range check.

Secure example

```
uint newRequired = uint(msg.value); if (newRequired <= m_maxConfirmations)
{ ... }
```

File 7: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.578

Source lines: 96-414

► Finding 7.1: Insecure File Upload (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash) {  
    // ...  
}
```

Explanation

The function `execute` accepts a file upload without type validation, allowing an attacker to send arbitrary files.

Impact

Remote code execution or data corruption

Entry point

/execute

HTTP methods

POST

Parameters

_data

Exploitation steps

- Send a malicious file as `\_data`

Example payloads

- `malicious_file.txt`

Conditions

Attacker has access to the `execute` function

Remediation

Validate and sanitize the `\_data` parameter

Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner {  
    require(bytes(_data).length > 0);  
    // ...  
}
```

File 8: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.560

Source lines: 1-61

► Finding 8.1: Unrestricted File Upload (High)

Vulnerable code

```
function play(uint number) payable{
  if (msg.value != 1 ether) throw;
  players[tot] = Player(msg.sender, number);
  tot++;
}
```

Explanation

The 'play' function does not validate the file type or size, allowing arbitrary files to be uploaded.

Impact

Remote code execution and data corruption

Entry point

/transfer

HTTP methods

POST

Parameters

number, players[tot]

Exploitation steps

- Upload a malicious file to the 'play' function.

Example payloads

- `malicious_file.exe`

Conditions

msg.value != 1 ether

Remediation

Validate the file type and size before uploading it.

Secure example

```
if (msg.value == 1 ether) {  
  players[tot] = Player(msg.sender, number);  
  tot++;  
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)